

摘要

整个方法采用“先建立可信边界—再冻结输入—再执行密码学门禁—最后跑攻击实验”的 fail-closed 策略。任何上游审计失败都禁止进入下一阶段；旧缓存不直接覆盖，而是归档；API 生成支持原子保存与断点续跑；不可构造的攻击样本记录为 skip/not_applicable，而不是临时替换为其他攻击；攻击条件下的回答结果独立于正常 Table VI，从而最大限度避免协议漂移、样本替换和结果混用。

最终 Stage 5，正式攻击计划总数为 1200，其中 applied=1124、skipped=76；300 个 benign controls 的 false alarm 为 0；benign Verify 共完成 30,000 次重复计时，Mean=1.212 ms、P50=1.091 ms、P95=2.893 ms。四大攻击族在所有 applicable trials 上的 Detection Rate 均为 100%。

报告结构

- 1. 修订背景、问题拆解与总体验证思路
- 2. Stage 1: 可信客户端上下文冻结与 D/Q 策略一致性审计
- 3. Stage 2: 查询改写信任边界与授权/解密路径修正
- 4. Stage 3: 60 条正式 API Rewrite Cache 的生成、冻结与验收
- 5. Stage 4: policy_digest 与 AES-GCM nonce 密码学一致性审计
- 6. Stage 5: 实验 G 正式修订协议、执行与最终结果
- 7. Stage 1–Stage 5 的跨阶段证据链与方法学分析
- 8. 论文写作可直接采用的结果与表述

1. 修订背景、问题拆解与总体验证思路

1.1 真正要解决的问题

旧实现把“语义理解”和“安全授权”混在了同一个 LLM 查询改写结构中。旧版 Rewrite 允许模型输出或间接保留 role、stage、course、max_sensitivity 等字段，这会产生检索效果或授权结果可能部分来自模型对权限属性的预测，而不是来自可信身份/属性来源。这样即使最终指标很好，也难以证明系统真正遵守了零信任和属性驱动的安全边界。

因此，修正是重新定义整个数据流：LLM 只负责语义信息；安全属性来自可信客户端；正式缓存把 semantic rewrite 与 trusted_context 分开；解密和授权只读取 trusted_context；正式实验开始前再验证本地策略哈希和 AES-GCM nonce；G 实验重新冻结攻击子类型、重复测量 benign Verify，并让 w/o Verification 真正走完后续解密、重排和回答链路。

1.2 总体方法：五道串联的正式门禁

阶段	核心目的	主要动作	控制原则
Stage 1	数据与策略可信基线	核对 164 文档、60 查询与 role_stage_policy；构建 trusted_client_context	不改 D/Q；不调用 API
Stage 2	代码级信任边界	LLM 只产出语义字段；安全属性只来自 trusted_context；旧缓存拒绝	14/14 单元测试 + deterministic smoke
Stage 3	正式语义输入冻结	对 60 查询调用 API，生成单模型、API-only、semantic-only 的正式缓存	可断点续跑；哈希冻结
Stage 4	密码学一致性门禁	逐 entry 核验 policy_digest；检查 (key_id, nonce) 唯一性	任一 mismatch/duplicate 立即 FAIL
Stage 5	G 正式实验	冻结 1200 攻击计划、300 benign、30,000 Verify 微基准、真实 downstream	最终 paper_ready_g=True

1.3 采用的八个方法学原则

- 信任边界分离：把语义增强与安全决策彻底拆开。LLM 只能提供 keywords、material_types、candidate_preferences；role、stage、course_membership、max_sensitivity 均来自可信客户端。
- Fail-closed：任何字段越界、缓存不兼容、哈希不一致、nonce 重复、上游 audit 非 PASS 都直接停止。
- 冻结优先：正式结果所依赖的查询、数据、trusted context、rewrite cache、攻击 subtype plan、answer subset 都在查看结果前冻结。
- 不覆盖原始数据：Stage 1/2/5 均避免覆盖 D/Q；旧缓存先归档。这样可追溯旧协议与新协议的差异。
- 可恢复 API 执行：Stage 3 和 Stage 5 Answers 都使用原子保存；API/网络/额度中断后直接续跑，避免重复调用和样本缺失。
- Applied-N 分母：预先计划的攻击如果因真实数据条件不可构造，记录 skip，不临时替换；Detection Rate 以 applied N 为分母。
- 高分辨率微基准：benign Verify 用 perf_counter_ns 等高分辨率计时，统一预热并固定重复次数；优先报告 P50/P95。
- 真实 downstream：w/o Verification 只关闭 proof Verify，其余策略、AES-GCM、BGE、Cross-Encoder、回答生成继续真实运行，观察实际损失而非假设后果。

2. Stage 1：可信客户端上下文冻结与 D/Q 策略一致性审计

2.1 目标与输入

Stage 1 的任务是先建立一个不依赖 LLM 的可信属性基线。脚本只读取 data\processed 中现有正式 D/Q，不调用 API，也不修改任何 D/Q 文件。正式输入为 Dpub、Dcourse、Dassess、Dgov 四类共 164 篇文档，以及 Qconcept、Qlookup、Qassess、Qmanage 四类共 60 条查询。四个查询类别各 15 条。

数据项	规模/分布
文档总数	164
Dpub / Dcourse / Dassess / Dgov	9 / 55 / 50 / 50
Sensitivity	public 10; internal 94; restricted 25; highly_restricted 35
查询总数	60
Qconcept / Qlookup / Qassess / Qmanage	15 / 15 / 15 / 15
单一课程	mit_6_5830_database_systems

所有正式文档必须使用 access_policy.role_stage_policy。Stage 1 不把“query 里写了什么”当成授权来源，而是逐个 relevant document 根据角色与阶段重新计算可见性，再与查询集中的 visible_documents、hidden_documents 和 answer_policy 对齐。

2.2 角色/阶段访问判定方法

阶段顺序被固定为 pre_class → in_class → assignment → pre_exam → current_exam → post_exam → course_archive。对文档 d、角色 u、查询阶段 s，若 u 不在 d.access_policy.role_stage_policy 中，则判为 role_restricted；若 u 存在但 s 早于该角色在文档策略中要求的开放阶段，则判为 stage_restricted；只有角色存在且阶段达到阈值时才判为 visible。

```
if role not in role_stage_policy:
    deny(reason="role_restricted")
elif stage_rank(query_stage) < stage_rank(required_stage_for_role):
    deny(reason="stage_restricted")
else:
    allow(reason="visible")
```

随后根据 relevant documents 中真正 visible 的数量重新推导回答策略：若全部 relevant 都可见，则 answer；若至少一个可见但仍有部分不可见，则 partial；若 relevant 存在但一个都不可见，则 refuse。原始查询中的 deny/reject/refuse 在审计中统一规范化为 refuse 语义。

2.3 max_sensitivity 的可信生成方法

针对 max_sensitivity 不能来自 LLM：冻结的本地 repository policy registry 一次性推导每个角色能接触到的最高 sensitivity ceiling，并将其作为“角色级上限”冻。这样 max_sensitivity 与查询文本无关、与模型输出无关。

Role	Frozen max_sensitivity	本地策略中该角色可访问的敏感级别分布
student	restricted	public 10; internal 59; restricted 2
ta	highly_restricted	public 10; internal 74; restricted 22; highly_restricted 20
instructor	highly_restricted	public 10; internal 89; restricted 25; highly_restricted 30
coordinator	highly_restricted	public 9; internal 35; restricted 3; highly_restricted 15
admin	highly_restricted	public 9; internal 20; restricted 3; highly_restricted 10

这里的 max_sensitivity 只是上限，不等于“达到该敏感级别就自动授权”。真正解密时仍必须同时满足 role_stage_policy、course membership 与 max_sensitivity 四类条件。因此它不会把角色授权模型简化为单一敏感等级。

2.4 trusted_context 的正式结构

```
{
  "role": "...",
  "stage": "...",
  "course_membership": ["mit_6_5830_database_systems"],
  "max_sensitivity": "..."
}
```

每条正式 query 都得到一个 trusted_context。role 和 stage 来自冻结的角色化评测元数据；course_membership 来自冻结的认证评测会话上下文；max_sensitivity 来自一次性本地策略注册表推导。该结构被写入 exp_protocol\trusted_client_context.json，同时生成 query_policy_alignment_audit.csv 和 trusted_client_context_audit.json。

2.5 Stage 1 的正式结果与意义

Stage 1 结果

STAGE 1 PASS; Documents=164; Queries=60; 单课程 membership 已冻结; 所有正式文档使用 role_stage_policy; query policy alignment 通过; LLM 未参与任何安全属性生成。

Stage 1 的最大价值是建立“可信属性的唯一来源”。从这一阶段开始，后面所有 rewrite、授权、解密、攻击实验都必须引用同一份 trusted_context，而不能再从 LLM 输出、自然语言 query 或旧缓存中的 filters 读取安全属性。

3. Stage 2: 查询改写信任边界与授权/解密路径修正

3.1 旧实现的问题

旧版 Rewrite 将语义字段与 filters 混在一起，模型可能返回 course、max_sensitivity 等内容；即使代码后处理覆盖了部分 role/stage，也存在 course 或 sensitivity 信息被模型保留的风险。这样会导致权限过滤结果受到非可信 LLM 输出影响。Stage 2 的目标是从 prompt、parser、cache schema、trusted context loader、authorization/decrypt 一整条链路同时封死这条路径。

3.2 新 Rewrite Schema: semantic-only

```
{
  "keywords": [...],
  "material_types": [...],
  "candidate_preferences": {
    "candidate_depth": 50,
    "final_top_k": 10
  }
}
```

允许的顶层字段只有 keywords、material_types、candidate_preferences。模型不再被要求输出 role、stage、course、course_membership、max_sensitivity、access_policy、permissions、filters 等任何安全字段。

3.3 两层越界防护：Prompt 禁止 + Parser 强拒绝

第一层是 prompt 明确声明“NEVER output, infer, copy, or request”安全字段。第二层更关键：parser 会递归扫描模型返回 JSON 的所有层级，只要发现 role、stage、course、course_membership、max_sensitivity、sensitivity、privacy、authorization、access_policy、permissions、filters、policy 等字段就直接抛出 ExperimentError。即使模型把安全字段嵌入子对象也无法绕过。

此外，parser 还检查 unexpected top-level field，并对 candidate_depth / final_top_k 做正整数和大关系校验。因此 Stage 2 不依赖模型“自觉遵守提示词”，而是通过程序约束建立 enforceable boundary。

3.4 API 请求最小化

正式 Rewrite API 的 user payload 只包含 query_id、原始 query 文本、query_category。role、stage、course membership、max sensitivity 不发送给模型。

```
user_payload = {
  "query_id": query["query_id"],
  "query": query["query"],
  "query_category": query["query_category"]
}
```

3.5 trusted_context 如何真正进入授权/解密

Stage 2 将 decrypt_if_authorized 改为显式接收 trusted_context，并按如下顺序执行：

1. 从 doc_tag 查本地 policy_registry；查不到立即拒绝。
2. 重新计算本地 policy_digest，与 entry.policy_digest 不一致则拒绝。
3. 执行 role_stage_policy(role, stage)；角色或阶段不满足则拒绝。
4. 检查文档 course 是否属于 trusted_context.course_membership。
5. 检查 document sensitivity 是否不高于 trusted max_sensitivity。
6. 根据 tau、doc_tag、policy_digest、version 派生预期 nonce；stored nonce 不等于 expected nonce 则拒绝。
7. 检查 ciphertext SHA-256。
8. 使用 AES-GCM(key, nonce, AAD) 解密；认证失败则拒绝。
9. 解密后再次核对 doc_id、course、sensitivity 与本地 registry 一致。

这使得 proof verification 之后的 decrypt/authorization 仍然拥有独立的策略和密码学约束，也是后来 Stage 5 中 F1 ciphertext Forge 即使跳过 proof verification 仍会被 AES-GCM 拦截的原因。

3.6 旧缓存处理与测试

Stage 2 不允许把旧 rewrite cache 当成新协议缓存继续使用。安装脚本会先备份/归档旧 formal query_rewrite_cache，然后执行新代码的 unit tests、compile 与 deterministic smoke。smoke 的作用不是形成论文结果，而是证明 60 条查询在不调用 API 的情况下都能通过新 schema、trusted_context 和正式 loader 的结构约束。

Stage 2 执行结果

本地执行 14/14 单元测试 PASS；60/60 deterministic smoke rewrite 通过；LLM security fields in cache = 0；trusted_context rows = 60/60；旧 formal cache 已归档而非复用。

4. Stage 3: 60 条正式 API Rewrite Cache 的生成、冻结与验收

4.1 为什么还需要 Stage 3

Stage 2 只是证明新信任边界可以工作，但 deterministic smoke 不能作为正式论文的语义改写结果。Stage 3 才真正调用外部 Rewrite API，为冻结的 60 条 query 生成正式 semantic rewrite，并将它们与 trusted_context 一起保存。

4.2 API 预检与模型一致性

Stage 3 从环境变量中读取 API base URL、API key 和 rewrite model。API key 只从环境变量读取，控制台只打印“使用了哪个环境变量名称”，绝不打印 key 值。若没有显式模型环境变量，则使用 experiment_g.json 中冻结的 default_model。配置的默认 Rewrite model 为 qwen3.7-plus、temperature=0、enable_thinking=false。正式运行时实际模型以 formal_rewrite_cache_audit.json 中记录的 rewrite_model 为唯一权威。

如果已有 cache：schema 不一致、source 非 API 或文件不可读时会归档；如果是同 schema、同模型的 partial API cache，则直接 resume；如果当前 cache 已经由另一个模型生成，则 Stage 3 明确 STOP，不允许静默混合或覆盖模型。

4.3 断点续跑与原子保存

每完成一条 query rewrite 就原子写入 cache，因此网络/配额中断不会导致已完成结果丢失。再次运行相同命令时会跳过已完成 query，仅补剩余部分。这一机制同时减少重复 API 成本，并避免因多次重跑导致 60 条 query 使用不同时间、不同模型或不同配置生成。

4.4 正式验收条件

- formal cache 恰好 60 rows；
- rewrite_source 只能是 api；
- 60 rows 只能出现一个 rewrite_model；
- 所有 row 的 rewrite_schema_version 必须一致；

- 禁止出现 filters、preferences、role、stage、course、course_membership、max_sensitivity 等旧安全字段；
- trusted_context 必须与 Stage 1 冻结值完全一致且 digest 一致；
- API key 字面值不得出现在 cache；
- 最后生成 formal_rewrite_cache_audit.json，并记录 cache SHA-256 与 trusted_context SHA-256。

Stage 3 状态

Stage 3 已 PASS。Stage 5 最终报告再次确认 stage3_pass=true，因此后续 G 使用的是通过正式 loader 和 audit 的新协议 rewrite cache，而不是 Stage 2 deterministic smoke 或旧协议缓存。

5. Stage 4: policy_digest 与 AES-GCM nonce 密码学一致性审计

5.1 设计目的

“索引 entry 中携带的 policy_digest 确实对应本地可信策略”，以及“同一 AES-GCM key 作用域内不存在 nonce 重复”。这是因为仅检查字段存在并不能证明索引与本地策略一致；而 AES-GCM 在同 key 下重复 nonce 会破坏安全假设。Stage 4 因此作为 G 之前的硬门禁。

5.2 policy_digest 审计

Stage 4 重建 protected index，并从冻结 D*.json 独立构造 local_by_tag。对每个 posting entry，使用与索引构建阶段完全相同的 canonical_json_bytes 对 document.access_policy 序列化，再计算 SHA-256。随后同时比较 local_policy_digest、protected_index.policy_registry 中的 registry digest 与 entry.policy_digest。

```
local_policy_digest = SHA256(canonical_json_bytes(local_document.access_policy))
PASS only if:
  local_policy_digest == entry.policy_digest
  registry_policy_digest == entry.policy_digest
```

此外还统计 missing local policy、doc_tag collision。任何一个 mismatch、missing 或 collision 都使 digest_gate=False，从而禁止进入正式 G。

5.3 nonce 唯一性审计

AES-GCM nonce 唯一性按教师要求在 (key_id, nonce) 作用域检查。为了不泄露真实 AES key，Stage 4 不导出 key，也不导出 key 的 hash；而是对公开的 per-entry key derivation context {tau, doc_tag, policy_digest, version} 计算非秘密 key_scope_id。固定 master key 下，该 context 唯一决定 entry key，因此可用于审计作用域标识。

每条 entry 记录 key_id、nonce_b64、nonce_length_bytes，以及同一 (key_id, nonce) 的出现次数。正式 gate 要求 duplicate pair count=0 且 nonce length 统一为 12 bytes。全局 nonce 是否重复只作为信息项；正式判定以同 key scope 为准。

Stage 4 状态

Stage 4 已 PASS；Stage 5 最终报告再次确认 `stage4_crypto_pass=true`。按 Stage 4 的 PASS 定义，这意味着 `policy_digest formal gate` 通过、`duplicate (key_id, nonce) pair count = 0`，且 AES-GCM nonce 长度为 12 bytes。

6. Stage 5：实验 G 正式修订协议、执行与最终结果

6.1 Stage 5 与实验 G

Stage 5 就是修订后的 Experiment G 正式执行阶段。Stage 1–Stage 4 都是为 G 和后续 H/I 建立可信输入与安全门禁；Stage 5 才真正执行四大攻击族、benign controls、Verify 微基准以及 w/o Verification 真实下游。它不会修改 E/F，也不会把 I 的 scalability 混入 G。

6.2 样本规模与攻击 subtype 冻结

正式规模保持 $60 \text{ queries} \times 5 \text{ fixed seeds} = 300 \text{ trials} / \text{attack family}$ 。Drop、Pollute、Forge、Hybrid 四类共 1,200 planned attacks；另有一套共享的 300 benign controls。subtype plan 在结果产生前冻结。若某 trial 因真实 posting 条件无法构造指定 subtype，则记录 `skip/not_applicable`；禁止临时改成其他 subtype，DR 只以 Applied N 为分母。

Attack Family	Subtype 分配	Planned N
Drop	D1–D6, 各 50	300
Pollute	P1–P5, 各 60	300
Forge	F1–F4 各 38; F5–F8 各 37	300
Hybrid	现有 registry H1: <code>drop_pollute_forge</code>	300
Total	四大类	1200

6.3 Detection Rate / FAR 的统计口径

对每个 applied attack trial，执行 Proposed verification。如果 Verify 返回失败，则说明该操纵被检测； $\text{Detection Rate} = \text{Detected N} / \text{Applied N}$ 。正常控制只使用同一组 300 benign responses 计算 False Alarm Rate，不把相同 benign control 重复当成四组独立样本。

这一口径很重要：76 个 skipped trial 并不是“验证没有检测到”，而是攻击前提在真实数据中不存在。例如 P5 需要能找到策略受限且位于请求 token 之外的条目，如果不存在就不能为了凑 N 人工替换。

6.4 Benign Verify 重复微基准

旧版本每个 trial 只计时一次，不足以支持亚毫秒结论。Stage 5 按修订协议先统一 warm-up 100 次，然后对 300 个 benign responses 各重复 Verify 100 次，共 30,000 次。计时边界只包围 `Verify(Tu,E(Tu),proof,commitment)`，不包含文件读取、JSON、proof generation、攻击注入、BGE、Cross-Encoder 或回答生成。

本实验同时包含 canonical、json_roundtrip、permute_token_response_order、reorder_object_fields、proof_regeneration 五种 benign 变体，每种 6,000 次，用于确认证验证开销不是特定序列化表示的偶然值。

6.5 w/o Verification 的真实 downstream

w/o Verification 不是“把 Detection Rate 设为 0”。它只跳过 proof Verify，之后的策略判断、AES-GCM、BGE 初排、Cross-Encoder 精排和回答生成继续真实执行。Stage 5 对三类机制不同的攻击执行 downstream：全部 applied Drop、全部 P3 append_foreign_valid_entry、全部 F1 forge_ciphertext。

- Drop：观察候选数量、Recall/nDCG、Top-10 overlap，以及固定回答子集的 action/Utility/GF/PVR。
- P3：观察外来合法 entry 是否通过策略/AES-GCM、是否进入候选和 Top-K、最终回答是否使用该 injected evidence。
- F1：观察跳过 proof 后 AES-GCM 是否认证失败、是否造成候选损失与 reranking/answer 变化。

6.6 回答层固定子集与双 Judge

回答层样本必须在看结果前声明。Stage 5 预计划 32 个 attack-answer cases：Drop 12（6 个 subtype 每个 2）、P3 10、F1 10。若预声明 case 无法形成可回答输入，则记录 skip，不补样本。Clean Proposed answer 对同一 query 只生成一次，并与攻击答案分别用 Judge A 与 Judge B 双判；攻击回答结果独立保存，不写入 H 正常 Table VI。

实际生成并完成双 Judge 的 attack answers 为 31，计划 32，其中 ready=31、skipped=1。由于若干 attack case 共享相同 query，clean baseline 只需生成 27 个不同 query 的答案。实际模型为 Generator=qwen3.7-plus、Judge A=qwen3.7-plus、Judge B=glm-5.2。

最终表格

6.7 G 总体验收结果

Attack Type	Planned N	Applied N	Skipped N	Detected N	Detection Rate (%)	Attack Verify Mean (ms)
Drop	300	297	3	297	100.00	0.475
Pollute	300	227	73	227	100.00	0.612
Forge	300	300	0	300	100.00	0.627
Hybrid	300	300	0	300	100.00	0.248
Overall	1200	1124	76	1124	100.00	0.483

Detection rate is computed over applicable attack trials. Trials that could not instantiate the pre-specified subtype were recorded as skipped rather than substituted. FAR was 0.00% (0/300) on the shared benign control set. Benign verification was independently microbenchmarked over 30,000 calls, yielding Mean=1.212 ms, P50=1.091 ms, and P95=2.893 ms.

四大攻击族对所有 applicable attacks 均实现 100% Detection Rate；总计 1124/1124 applied manipulations 被检测。300 个 benign controls 中 false alarms=0，因此 FAR=0%。

解释 attack verify mean 与 benign P50/P95 的差异

攻击条件下很多操纵会在早期检查处立即失败，例如 D5 的 query_token_set_mismatch，因此 attack Verify 平均时间可能远低于 benign 的完整成功验证。两者不是同一运行路径，不能用 attack mean 代替 benign 成功验证开销。

6.8 Benign Verify 正式微基准结果

Variant	N	Mean (ms)	P50 (ms)	P95 (ms)	Min	Max
Overall	30000	1.212	1.091	2.893	0.240	11.269
canonical	6000	1.227	1.074	2.951	0.245	11.269
json_roundtrip	6000	1.243	1.095	2.933	0.242	8.185
permute_token_response_order	6000	1.204	1.080	2.971	0.242	7.065
reorder_object_fields	6000	1.192	1.087	2.869	0.240	6.160
proof_regeneration	6000	1.194	1.108	2.756	0.241	5.483

最重要的变化是：正式 benign Verify P50=1.091 ms、P95=2.893 ms，因此论文应写“millisecond-level verification overhead”，不应继续沿用旧协议下“约 0.3 ms / sub-millisecond”的笼统表述。

6.9 Drop/P3/F1 downstream 检索影响

Attack	N	Clean cand.	w/o Verif cand.	Δ cand.	Clean Recall	Attack Recall	Δ Recall	Clean nDCG	Attack nDCG	Δ nDCG	Top-K Jaccard
Drop	297	14.684	14.259	-0.424	0.960	0.948	-0.012	0.835	0.824	-0.011	0.921
P3 cross-token injection	60	14.550	14.967	+0.417	0.960	0.960	+0.000	0.838	0.838	+0.000	0.901
F1 ciphertext forge	38	15.211	14.789	-0.421	0.989	0.872	-0.117	0.884	0.796	-0.088	0.901

6.10 P3 合法跨-token 注入的安全含义

P3 共 60/60 applied。Proposed 路径在 Verify 阶段全部检测并停止。若跳过 Verification，直接注入条目中 39/60 成功通过授权并解密，21/60 被 role_stage_denied 阻止。成功解密后，有 21 个明确进入 Top-K，16 个明确未进入 Top-K；其余记录没有可定义的进入状态。

值得注意的是，P3 的平均 Recall 与 nDCG Δ 都为 0。这并不表示攻击无风险，而是说明“传统 relevance metric 不一定能感知结构上合法、语义上可能无害但来源不合法的跨-token 注入”。在回答子集中还出现 1 个回答实际采用 injected evidence 的 case，说明完整性验证的价值不能仅用 Recall/nDCG 衡量。

6.11 F1 ciphertext Forge: Verification 与 AES-GCM 的互补性

F1 共 38/38 applied。Proposed 全部在 Verify 阶段检测。w/o Verification 下，直接 forged entry 解密成功数为 0/38：其中 28 个因 AES-GCM authentication failed，被策略提前拒绝的 role_stage_denied 为 10。

虽然伪造密文没有被当作合法明文接受，但关闭 Verification 仍造成候选损失和检索下降：平均 candidate $\Delta=-0.421$ ，Recall $\Delta=-0.117$ ，nDCG $\Delta=-0.088$ 。这说明 proof verification 与 AES-GCM 不是重复防线：前者在语义处理前阻止结构/承诺不一致；后者在跳过 proof 后仍能保护密文真实性，但系统可能已经因为异常条目被丢弃而失去候选。

6.12 Drop: 删除攻击的实际影响

Drop 的 297 个 applied trials 中，平均候选数下降 -0.424，Recall $\Delta=-0.012$ ，nDCG $\Delta=-0.011$ ，Top-K Jaccard=0.921。平均影响看似有限，是因为很多被删 entry 本身不影响最终 Top-K 或相关文档；但 D5 drop_whole_token_response 的影响明显更大。

D5 的平均 Recall $\Delta=-0.068$ ，nDCG $\Delta=-0.065$ ，说明当整个 token response 被省略时，关闭 Verification 会产生可测的 retrieval utility 损失。

6.13 31 个攻击回答的双 Judge 结果

Target	N_answer	Clean Utility	Attack Utility	Δ Utility	Clean GF	Attack GF	Δ GF	Clean PVR	Attack PVR	Δ PVR
Drop	11	0.489	0.523	+0.034	0.800	0.697	-0.103	0.182	0.182	+0.000
P3	10	0.550	0.588	+0.037	0.750	0.771	+0.021	0.100	0.100	+0.000
F1	10	0.438	0.500	+0.062	0.752	0.760	+0.007	0.100	0.200	+0.100
Overall	31	0.492	0.536	+0.044	0.769	0.741	-0.027	0.129	0.161	+0.032

总体上 Attack Utility 比 Clean 高 0.044，但 Grounding Fidelity 下降 0.027，PVR 上升 0.032。这里不能选择性地只报告“变差”的指标：Utility 小幅上升是真实 Judge 输出，可能来自攻击条件下回答更直接、更长或更符合表面任务需求；但安全性需要结合 GF、PVR、action 和 injected evidence use 解释。

尤其是 F1 回答组中 Attack PVR 相比 Clean 上升 0.100；P3 组虽然平均 GF 没下降、PVR 没上升，但确实存在 1 个回答使用 injected evidence 的 case。这再次说明平均指标之外还必须保留机制级 case evidence。

6.14 Stage 5 最终门禁

最终 final_validation_report 还确认 attack answers 与正常 Table VI 分离，normal_table_vi_modified=false。这意味着 G 的攻击回答可以作为 H 的攻击补充证据，但不能直接代替或污染 H 的正常 60×5 主表。

6.15 Stage 5 攻击子类型完整结果

ID	Family	Subtype	Planned	Applied	Skipped	DR %	P50 ms	P95 ms
D1	Drop	drop_first_entry	50	50	0	100.00	0.299	1.441
D2	Drop	drop_middle_entry	50	50	0	100.00	0.392	1.610
D3	Drop	drop_last_entry	50	50	0	100.00	0.292	1.585
D4	Drop	drop_multiple_entries	50	48	2	100.00	0.228	1.878
D5	Drop	drop_whole_token_response	50	50	0	100.00	0.007	0.013
D6	Drop	drop_and_recompute_local_manifest	50	49	1	100.00	0.627	1.888
P1	Pollute	append_visible_distractor	60	47	13	100.00	0.302	1.859
P2	Pollute	replace_with_visible_distractor	60	47	13	100.00	0.356	1.865
P3	Pollute	append_foreign_valid_entry	60	60	0	100.00	0.590	2.157

ID	Family	Subtype	Planned	Applied	Skipped	DR %	P50 ms	P95 ms
P4	Pollute	duplicate_committed_entry	60	60	0	100.00	0.312	1.633
P5	Pollute	append_policy_hidden_entry	60	13	47	100.00	0.303	1.278
F1	Forge	forge_ciphertext	38	38	0	100.00	0.552	1.725
F2	Forge	forge_doc_tag	38	38	0	100.00	0.518	1.524
F3	Forge	forge_entry_tau	38	38	0	100.00	0.393	1.242
F4	Forge	forge_ordinal	38	38	0	100.00	0.361	1.339
F5	Forge	forge_manifest_count	37	37	0	100.00	0.405	1.478
F6	Forge	forge_manifest_root	37	37	0	100.00	0.386	1.731
F7	Forge	forge_global_path	37	37	0	100.00	0.440	1.524
F8	Forge	replay_other_token_proof	37	37	0	100.00	0.702	2.271
H1	Hybrid	drop_pollute_forge	300	300	0	100.00	0.130	0.976

ID	Subtype	Skipped N	Skip reason
D4	drop_multiple_entries	2	{"no_posting_with_at_least_2_entries": 2}
D6	drop_and_recompute_local_manifest	1	{"no_posting_with_at_least_2_entries": 1}
P1	append_visible_distractor	13	{"no_visible_distractor_entry_outside_requested_tokens": 13}
P2	replace_with_visible_distractor	13	{"no_visible_distractor_entry_outside_requested_tokens": 13}
P5	append_policy_hidden_entry	47	{"no_policy_hidden_distractor_entry_outside_requested_tokens": 47}

所有 20 个实际攻击子类型在其 Applied N 上 DR 均为 100%。Skip 主要集中在 Pollute P5（47 个）以及 P1/P2（各 13 个），另有 Drop D4=2、D6=1；这些 skip 反映真实索引中目标 posting/distractor 条件并非对所有 query 都存在，而不是漏检。

7. Stage 1–Stage 5 的跨阶段证据链与方法学分析

7.1 为什么这不是五个相互独立的小脚本

Stage 1–Stage 5 实际上构成一个单向证据链：Stage 1 决定可信安全属性；Stage 2 把代码实现强制对齐这个信任边界；Stage 3 冻结正式语义改写输入；Stage 4 证明受保护索引中的策略摘要与 nonce 满足密码学一致性；Stage 5 才在同一输入与同一安全边界上测攻击检测、验证开销和 downstream。

上游证据	下游依赖	若缺失会造成的问题
trusted_client_context	Stage2/3/5 authorization	LLM 或 query text 可能重新控制 role/stage/course/sensitivity
formal rewrite cache	Stage5 query terms	不同实验可能使用不同语义解析，导致不可比较
policy_digest audit	Stage5 protected entries	无法证明 entry 策略摘要与本地真实策略一致
nonce uniqueness audit	AES-GCM entries	nonce 重复会破坏 AEAD 安全假设
frozen subtype plan	Stage5 DR	可产生按结果挑攻击/替换 subtype 的选择偏差
frozen answer plan	Stage5 downstream answer	可产生只挑最明显攻击样本的事后选择偏差

7.2 可信属性与语义改写的“最小耦合”

修订后的核心结构可以概括为 $Q_sem = LLM(Q_nl)$, $C_u = \text{trusted client attributes}$, $Q_str = Q_sem UC_u$ 。但只表示在后续检索流程中同时使用，不表示模型可以编辑 C_u 。**semantic rewrite** 用于提高检索语义表达；**trusted_context** 用于限定允许访问的证据空间。两者职责分离后，才能把“检索效果提升”与“权限预测”解耦。

7.3 多层防线并非重复

Stage 5 F1 结果非常清楚地展示了 **proof verification** 与 **AES-GCM** 的互补性：**Proposed** 可以在 **Verify** 阶段直接阻断 **ciphertext forge**；如果关闭 **Verify**，**AES-GCM** 仍拒绝篡改密文，因此 **forged content** 不会被接受。但候选会损失，**Recall/nDCG** 仍下降。这说明可验证检索的目标不仅是“最终不泄密”，还包括“在语义处理前发现服务端操纵并保护候选完整性”。

7.4 传统 IR 指标不足以覆盖完整性风险

P3 是第二个关键案例。P3 的平均 **Recall/nDCG** 不变，但 39/60 外来合法条目可以在跳过 **Verify** 后通过授权/解密，其中 21 个进入 **Top-K**，且有 1 个回答实际使用 **injected evidence**。说明 **relevance metric** 只回答“是否检到了相关文档”，不能回答“这个证据是否来自承诺的正确 **token response**”。因此 **G** 的完整性指标与 **H** 的 **grounding/policy** 指标是必要补充，而不是重复评价。

7.5 为什么要同时报告 **Planned / Applied / Skipped**

若只报告“300 trial/类”，会把无法构造的攻击也错误地纳入分母；若为了保证每类都有 300 **applied** 而临时换 **subtype**，又会破坏预冻结协议。当前做法保留 **planned=1200**，同时诚实报告 **applied=1124**、**skipped=76**，并公开 **skip reason**。这样既保留原定实验规模，也避免人为伪造适用条件。

7.6 为什么 **benign** 微基准要独立于攻击验证时间

攻击验证时间高度依赖失败位置：**query_token_set_mismatch** 可以极早退出，而 **global_manifest_proof_failed** 需要走到更深的 **proof** 链。若把这些 **attack times** 与 **benign** 成功验证混在一起平均，会得到没有明确语义的数字。因此论文应该用独立 **benign microbenchmark** 表示正常部署开销，用 **subtype P50/P95** 表示不同异常的检测成本。

8. 论文写作可直接采用的结果与表述

在修订后的正式可验证性实验中，我们对 60 条角色化查询和 5 个固定随机种子构成的攻击计划进行评估。四类攻击共计划 1,200 个 **trial**，其中 1,124 个满足预定义攻击前提并成功构造，76 个因真实索引条件不满足而按协议记录为 **skipped**，未进行子类型替换。在全部 1,124 个 **applicable manipulations** 上，**Proposed Framework** 均在语义处理前检测到异常，整体 **Detection Rate** 为 100%，同时 300 个 **benign controls** 未出现误报（**FAR=0%**）。

对正常响应的 30,000 次重复 **Verify** 微基准显示，验证开销处于毫秒级，**Mean** 为 1.212 ms，**P50** 为 1.091 ms，**P95** 为 2.893 ms。由于攻击响应可能在较早的一致性检查处失败，其检测时间通常低于完整 **benign verification**，因此本文分别报告攻击子类型的验证时间与正常路径微基准，避免将早停时间误解为正常验证开销。

在关闭 proof verification 但保留策略判断、AES-GCM、BGE 和 Cross-Encoder 的真实下游实验中，Drop 导致平均候选数、Recall 和 nDCG 下降；F1 ciphertext Forge 虽然全部被 AES-GCM 或策略层阻断而未直接解密成功，但仍造成候选损失及 Recall/nDCG 下降；P3 合法跨-token 注入则显示，传统 Recall/nDCG 可能保持不变，但部分外来合法 entry 能通过后续授权/解密并进入 Top-K，且个别回答实际采用了注入证据。该结果表明，检索完整性验证与加密认证、访问控制和语义相关性指标承担不同职责，完整系统需要在语义处理前显式验证服务器返回是否与承诺索引状态一致。

Under the revised formal protocol, we evaluated 60 role-aware queries with five fixed seeds, yielding 1,200 planned manipulation trials. Of these, 1,124 trials satisfied the pre-specified attack preconditions and were successfully instantiated, while 76 were recorded as skipped without subtype substitution. The proposed verifier detected all 1,124 applicable manipulations across Drop, Pollute, Forge, and Hybrid attacks, resulting in a 100% detection rate, with no false alarms on 300 benign controls (FAR = 0%).

A repeated microbenchmark over 30,000 benign Verify calls yielded a mean verification latency of 1.212 ms, with P50 and P95 latencies of 1.091 ms and 2.893 ms, respectively. Attack-side verification times are reported separately because manipulated responses often trigger early termination at different consistency checks and therefore do not represent the cost of a full successful verification path.

The downstream ablation further demonstrates that verification is not redundant with authorization or authenticated encryption. When proof verification was disabled, omission attacks reduced candidate coverage and retrieval effectiveness, whereas forged ciphertexts were still rejected by policy checks or AES-GCM authentication but caused candidate loss and measurable Recall/nDCG degradation. Valid cross-token injection was particularly informative: conventional Recall/nDCG could remain unchanged even when foreign committed entries passed later authorization/decryption and entered the Top-K, and one evaluated answer used injected evidence. These results show that retrieval integrity must be evaluated separately from semantic relevance and confidentiality.